Firebase Push Notifications

Flutter Integration Guide - Dovix AI Employee App

Written for: the Flutter developer integrating push notifications into the mobile app.

Overview

The backend is fully wired up: every notification event listed below now sends a push through Firebase Cloud Messaging (FCM) automatically, no further backend work needed. What's left is entirely on the app side - registering this device to receive pushes, and handling them when they arrive.

Events that trigger a push today:

- **Tasks** - assigned to you (from the admin Task Board, a project, or a team board) or reassigned to you.
- **Leave** - your request approved or rejected.
- **Face ID update** - your re-registration request approved or rejected.
- **Loan** - your request approved or rejected.
- **Reimbursement** - your request approved or rejected.
- **Assets** - one assigned to you, or a return requested.
- **Support tickets** - yours marked resolved.
- **ID card** - yours generated and ready.
- **Company policy** - a new rule published (sent to everyone in that office/branch).

That list is everything currently wired on the backend - if a new event type gets added later, it goes through the exact same mechanism and needs no changes on the Flutter side; only the `type`/`deepLink` values in the payload change (section 3.5).

What's already live on the backend:

- Sending pushes via FCM for every event above.
- An endpoint to register this device's FCM token.
- Endpoints to list notifications, mark them read, and get the unread badge count.

What you need to build in Flutter:

- Add the Firebase config file(s) to the project.
- Initialize Firebase and request notification permission.
- Get the device's FCM token and register it with our API.
- Handle a token refresh the same way.
- Show/handle notifications: foreground, background, and terminated-app taps.

1. Files You Need From Us

google-services.json - Android config, already generated. Place it at `android/app/google-services.json` in the Flutter project.

GoogleService-Info.plist - only if the app also targets iOS. Ask for this if it hasn't been generated yet (Firebase Console → iOS app → download); place it via Xcode into the Runner target (drag into `ios/Runner/`, make sure "Copy items if needed" is checked).

2. Packages (pubspec.yaml)

```
dependencies:
  firebase_core: ^3.x
  firebase_messaging: ^15.x
  flutter_local_notifications: ^18.x # shows the banner while app is in foreground
  http: ^1.x # or whatever this app already uses for API calls
```

Run the FlutterFire CLI (`flutterfire configure`) if this project doesn't already have Firebase set up, or add the Gradle/CocoaPods plugin lines manually per the standard FlutterFire install docs - either way, the two config files above are what tie it to *this* Firebase project.

3. Setup Steps

3.1 Initialize Firebase

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}
```

3.2 Request permission and get the token (after login)

```
final messaging = FirebaseMessaging.instance;
await messaging.requestPermission(alert: true, badge: true, sound: true);

final fcmToken = await messaging.getToken();
await registerDeviceWithBackend(fcmToken); // see 3.3

// Call registerDeviceWithBackend again whenever this fires - Firebase can
// rotate the token at any time, not just on first install:
messaging.onTokenRefresh.listen(registerDeviceWithBackend);
```

3.3 Register the device with our API

Call this once right after login (you already have the JWT `accessToken` from the login response at that point), and again on every token refresh.

```
POST {baseUrl}/api/v1/notifications/register-device
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "deviceId": "<stable id - see note below>",
  "fcmToken": "<token from getToken()>",
  "platform": "ANDROID" // or "IOS"
}
```

deviceId must be stable across app restarts - generate it once (e.g. with the `device_info_plus` or `platform_device_id` package, or a UUID you persist locally) and reuse it every time. Re-registering the same `deviceId` just updates its token; a new random one each launch would create duplicate device rows on our side.

An employee can have several devices registered at once (phone + tablet) - every one of them receives every push, so no special handling needed for multi-device.

3.4 Handle incoming notifications

Three states to cover:

- **Foreground** (app open) - FCM does not show a system banner automatically; listen and show one yourself with `flutter_local_notifications`.
- **Background** (app minimized) - the OS shows the system notification automatically; you only need to handle the tap.
- **Terminated** (app was killed) - same as background, but read the initial message on cold start.

```
// Foreground - show it ourselves
FirebaseMessaging.onMessage.listen((RemoteMessage message) {
  showLocalNotification(message.notification?.title, message.notification?.body);
});

// Tapped while backgrounded
FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
  handleNotificationTap(message.data);
});

// App was fully closed, opened via a notification tap
final initialMessage = await FirebaseMessaging.instance.getInitialMessage();
if (initialMessage != null) {
  handleNotificationTap(initialMessage.data);
}
```

Note: a top-level (non-class) function annotated `@pragma('vm:entry-point')` is also required for background taps per FlutterFire's own setup - see their “background messages” docs if this is new to the project.

3.5 Use the data payload to navigate

Every push carries a `data` payload with the same shape as the notification-list API below, so one model/handler covers both:

Field	Example	Meaning
`notificationId`	`ntf_42`	Same id format as GET /notifications
`type`	`LEAVE`	One of: ATTENDANCE, LEAVE, DOCUMENT, SHIFT, REPORT
`deepLink`	`/leave/requests/`	Route to open on tap (may be empty)

Map `type` to a screen (e.g. LEAVE → leave requests screen) and use `deepLink` if you want finer routing than the type alone gives you.

4. API Reference - Notifications

All endpoints below require `Authorization: Bearer <accessToken>` (the same JWT the rest of the app already uses) and return the standard envelope `{success, data, message, error}`. Full request/response examples for each are already in api/docs (Notifications section) - this table is just the quick map.

Method & Path	Purpose
`POST /api/v1/notifications/register-device`	Register/update this device's FCM token (step 3.3)
`GET /api/v1/notifications`	List notifications, paginated, newest first, includes unreadCount
`PATCH /api/v1/notifications/{id}/read`	Mark one notification read
`POST /api/v1/notifications/mark-all-read`	"Mark all read" action
`GET /api/v1/notifications/unread-count`	Cheap poll for the badge count on screen focus

5. Testing Checklist

1. Log in, confirm `POST /notifications/register-device` fires and returns success.
2. Ask an admin to trigger a real event (assign you a task, approve/reject a leave request) and confirm a push arrives - foreground, backgrounded, and fully closed.
3. Tap the push in each of those three states and confirm it opens the app to the right screen via `data.deepLink`/`data.type`.
4. Kill and reopen the app; confirm the token is only re-registered when it actually changes (no need to call the endpoint on every launch, only after `getToken()`/`onTokenRefresh`).
5. Confirm `GET /notifications/unread-count` matches the badge shown after a new push arrives.

Questions on the backend side (payload shape, adding a new notification type, timing) - ask the backend developer; this document covers only what's needed in the Flutter app itself.